Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет «Информатика и системы управления»

Кафедра «Системы обработки информации и управления»

Отчет по лабораторной работе №3

«Подготовка обучающей и тестовой выборки, кросс-валидация и подбор гиперпараметров на примере метода ближайших соседей»

по дисциплине «Технологии машинного обучения»

Выполнил:
студент группы ИУ5Ц-83Б
Пронин В.К.
подпись, дата

Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата

2026 г.

СОДЕРЖАНИЕ ОТЧЕТА

1. Цель лабораторной работы 3
2. Задание 3
3. Основные характеристики датасета 3
4. Листинг программы 4
5. Предобработка данных 5
6. Подготовка признаков и целевой переменной 8
7. Модель с произвольным K=5 10
8. Подбор гиперпараметров с GridSearchCV и KFold 10
9. Подбор гиперпараметров с GridSearchCV и ShuffleSplit 11
10. Подбор гиперпараметров с RandomizedSearchCV 12
11. Сравнение результатов 12
12. Вывод 15

1. Цель лабораторной работы

Изучение способов подготовки выборки и подбора гиперпараметров на примере метода ближайших соседей (KNN).

2. Задание

1. Выбрать набор данных для решения задачи регрессии.
2. Провести удаление или заполнение пропусков и кодирование категориальных признаков.
3. Разделить выборку на обучающую и тестовую с помощью train_test_split.
4. Обучить модель KNN с произвольным K и оценить качество.
5. Произвести подбор гиперпараметров с использованием GridSearchCV и RandomizedSearchCV и двух стратегий кросс-валидации.
6. Сравнить метрики качества исходной и оптимальной моделей.

3. Основные характеристики датасета

Название: Used Cars Dataset

Источник: Kaggle (https://www.kaggle.com/datasets/shivanink8/used-cars-dataset)

Описание:

Датасет содержит информацию о 52 подержанных автомобилях из региона Сиэтла, собранную с сайтов autotrader.com и carfax.com. Данные включают технические характеристики автомобилей и историю обслуживания.

Размер датасета: 52 строки × 27 столбцов

Структура данных:

Признак	Тип	Описание
id	int64	Идентификатор
brand	object	Марка автомобиля
model	object	Модель
year	int64	Год выпуска
miles	int64	Пробег (мили)
city_mileage	int64	Расход в городе (миль/галлон)
highway_mileage	int64	Расход на шоссе (миль/галлон)
horsepower	int64	Мощность (л.с.)
torque	int64	Крутящий момент
engine_capacity_litre	float64	Объем двигателя (л)
fuel_capacity	float64	Объем топливного бака
num_cylinder	int64	Количество цилиндров
num_seat	int64	Количество мест
num_owners	int64	Количество владельцев
price	int64	Цена (целевая переменная)
link	object	Ссылка на объявление

condition	float64	Состояние
type	object	Тип кузова
doors	int64	Количество дверей
wheel_drive	int64	Привод
engine_type	object	Тип двигателя
speed_levels	float64	Количество передач
front_headroom	float64	Пространство для головы спереди
front_legroom	float64	Пространство для ног спереди
rear_headroom	float64	Пространство для головы сзади
rear_legroom	float64	Пространство для ног сзади
service_records	int64	Количество записей об обслуживании

Целевая переменная: price - стоимость автомобиля в долларах.

4. Листинг программы

```
# Подключение необходимых библиотек
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# Для машинного обучения
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV, KFold, ShuffleSplit
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.compose import ColumnTransformer

# Загрузка данных
df = pd.read_csv('used_cars.csv')
print(f'Размер датасета: {df.shape}')
display(df.head())
```

Рисунок 1 - Загрузка необходимых библиотек и вывод размера датасета

Результат:

Размер датасета: (52, 27)

	id	brand	model	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	...	type	doors	wheel_drive	engine_type	spe
0	3	Chevrolet	Volt LT	2019	27173	43	42	149	294	1.5	...	sedan	4	2	hybrid_gas_electric	
1	6	Hyundai	Elantra SE	2017	76941	29	38	146	132	2.0	...	sedan	4	2	gas	
2	7	Ford	Focus	2014	97027	27	37	159	146	2.0	...	sedan	4	2	gas	
3	8	Honda	Civic	2016	95396	31	42	158	138	1.5	...	sedan	4	2	gas	
4	9	Hyundai	Elantra SE	2019	28405	29	38	147	132	2.0	...	sedan	4	2	gas	

5 rows × 27 columns

Рисунок 2 - Первые 5 строк датасета

```
# Информация о данных
df.info()
```

Рисунок 3 - Информация о данных (df.info())

Результат:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52 entries, 0 to 51
Data columns (total 27 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   id                     52 non-null     int64
 1   brand                  52 non-null     object
 2   model                  52 non-null     object
 3   year                   52 non-null     int64
 4   miles                  52 non-null     int64
 5   city_mileage           52 non-null     int64
 6   highway_mileage        52 non-null     int64
 7   horsepower             52 non-null     int64
 8   torque                 52 non-null     int64
 9   engine_capacity_litre  52 non-null     float64
 10  fuel_capacity          52 non-null     float64
 11  num_cylinder           52 non-null     int64
 12  num_seat               52 non-null     int64
 13  num_owners             52 non-null     int64
 14  price                  52 non-null     int64
 15  link                   52 non-null     object
 16  condition              1 non-null      float64
 17  type                   52 non-null     object
 18  doors                  52 non-null     int64
 19  wheel_drive            52 non-null     int64
 20  engine_type            52 non-null     object
 21  speed_levels           51 non-null     float64
 22  front_headroom         52 non-null     float64
 23  front_legroom          52 non-null     float64
 24  rear_headroom          52 non-null     float64
 25  rear_legroom           52 non-null     float64
 26  service_records        52 non-null     int64
dtypes: float64(8), int64(14), object(5)
memory usage: 11.1+ KB
```

```
# Статистическое описание
df.describe()
```

	id	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	fuel_capacity	num_cylinder	...	pr
count	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	...	52.000(
mean	31.365385	2018.673077	33901.250000	29.038462	37.423077	176.865385	177.923077	1.832692	14.232692	4.038462	...	21763.423(
std	15.378912	1.396370	22700.646139	5.947356	4.136619	33.793507	47.256433	0.379743	2.603973	0.277350	...	3532.455:
min	3.000000	2014.000000	5000.000000	17.000000	24.000000	143.000000	99.000000	1.400000	7.000000	4.000000	...	13590.000(
25%	18.750000	2018.000000	16454.750000	26.500000	35.750000	152.000000	138.000000	1.500000	12.400000	4.000000	...	19519.000(
50%	31.500000	2019.000000	27448.500000	30.000000	38.000000	159.500000	181.000000	2.000000	13.600000	4.000000	...	21998.500(
75%	44.250000	2019.250000	42442.000000	30.000000	40.000000	192.000000	192.000000	2.000000	15.050000	4.000000	...	22996.000(
max	57.000000	2022.000000	97027.000000	55.000000	49.000000	288.000000	294.000000	3.500000	19.000000	6.000000	...	33777.000(

8 rows × 22 columns

Рисунок 4 - Статистическое описание числовых признаков (df.describe())

5. Предобработка данных

Анализ пропущенных значений

```python
# Проверка пропущенных значений
print('Пропущенные значения:')
print(df.isnull().sum())

# Визуализация пропусков
plt.figure(figsize=(10, 6))
missing_data = df.isnull().sum()
missing_data = missing_data[missing_data > 0]
missing_data.plot(kind='barh')
plt.title('Количество пропущенных значений')
plt.xlabel('Количество')
plt.tight_layout()
plt.show()
```

Рисунок 5 - Код проверки пропущенных значений

Результат:

```
Пропущенные значения:
id                        0
brand                     0
model                     0
year                      0
miles                     0
city_mileage              0
highway_mileage           0
horsepower                0
torque                    0
engine_capacity_litre     0
fuel_capacity             0
num_cylinder              0
num_seat                  0
num_owners                0
price                     0
link                      0
condition                51
type                      0
doors                     0
wheel_drive               0
engine_type               0
speed_levels              1
front_headroom            0
front_legroom             0
rear_headroom             0
rear_legroom              0
service_records           0
dtype: int64
```

Рисунок 6 - Результат проверки пропущенных значений

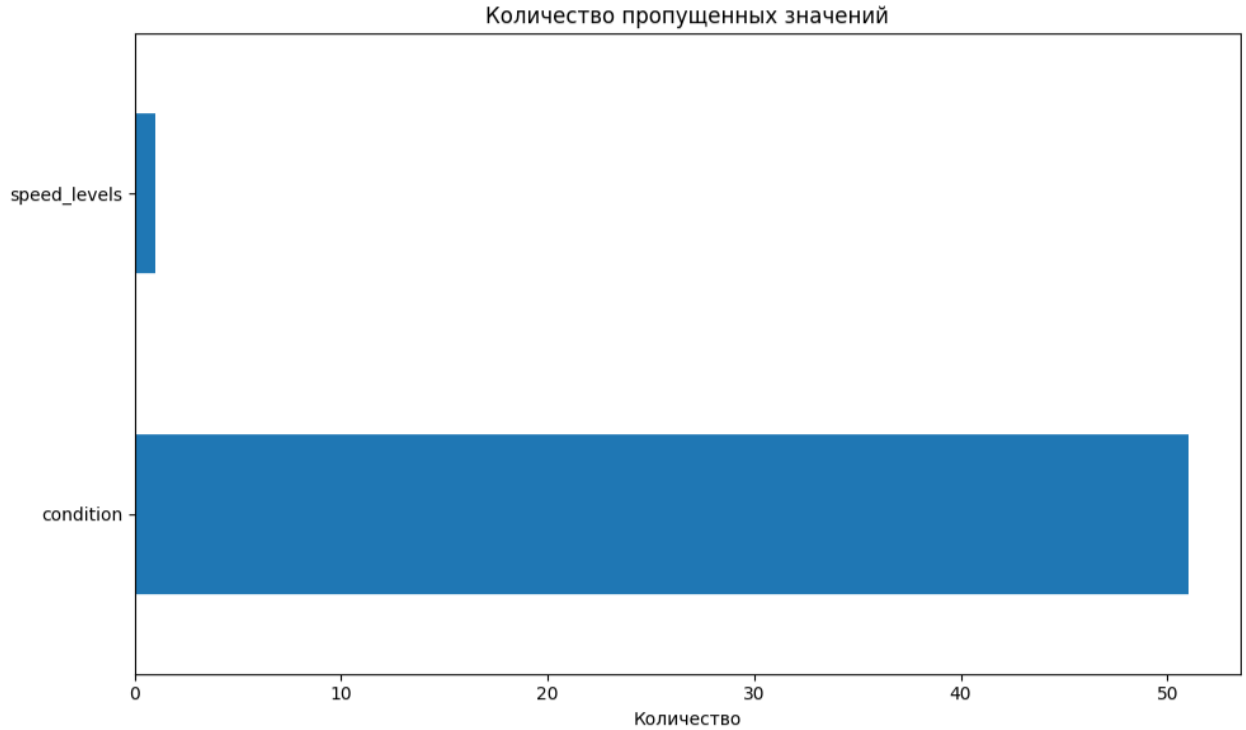

Рисунок 7 - Визуализация пропущенных значений (гистограмма)

Вывод: Обнаружены пропуски в столбцах condition (51 пропуск) и speed_levels (1 пропуск).

Заполнение пропусков

```
# Заполнение пропусков
for col in df.columns:
    if df[col].isnull().sum() > 0:
        if df[col].dtype in ['int64', 'float64']:
            # Для числовых - медиана
            df[col].fillna(df[col].median(), inplace=True)
            print(f'Заполнены пропуски в {col} медианой')
        else:
            # Для категориальных - мода
            df[col].fillna(df[col].mode()[0], inplace=True)
            print(f'Заполнены пропуски в {col} модой')

print('\nПосле заполнения:')
print(df.isnull().sum())
```

Рисунок 8 - Код заполнения пропусков

Результат:

```
Заполнены пропуски в condition медианой
Заполнены пропуски в speed_levels медианой

После заполнения:
id                      0
brand                   0
model                   0
year                    0
miles                   0
city_mileage            0
highway_mileage         0
horsepower              0
torque                  0
engine_capacity_litre   0
fuel_capacity           0
num_cylinder            0
num_seat                0
num_owners              0
price                   0
link                    0
condition               0
type                    0
doors                   0
wheel_drive             0
engine_type             0
speed_levels            0
front_headroom          0
front_legroom           0
rear_headroom           0
rear_legroom            0
service_records         0
dtype: int64
```

Рисунок 9 - Результат заполнения пропусков

Проверка дубликатов и анализ категориальных признаков

```python
# Проверка дубликатов
print(f'Количество дубликатов: {df.duplicated().sum()}')
if df.duplicated().sum() > 0:
    df = df.drop_duplicates()

# Анализ уникальных значений в категориальных признаках
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    print(f'{col}: {df[col].nunique()} уникальных значений')
    if df[col].nunique() == 1:
        print(f'  -> Признак {col} имеет только одно значение, удаляем')
        df = df.drop(col, axis=1)
```

Рисунок 10 - Код проверки дубликатов и анализа категориальных признаков

Результат:

```
Количество дубликатов: 0
brand: 6 уникальных значений
model: 15 уникальных значений
link: 51 уникальных значений
type: 1 уникальных значений
  -> Признак type имеет только одно значение, удаляем
engine_type: 2 уникальных значений
```

Рисунок 11 - Результат проверки дубликатов и удаления признака type

6. Подготовка признаков и целевой переменной

```python
# Определение признаков и целевой переменной
target_col = 'price'
drop_cols = ['price', 'link', 'id']  # id и link не несут полезной информации

X = df.drop(columns=drop_cols, errors='ignore')
y = df[target_col]

print(f'Признаки: {X.shape[1]}')
print(f'Целевая переменная: {y.name}')
```

Рисунок 12 - Определение признаков и целевой переменной

Результат:

```
Признаки: 23
Целевая переменная: price
```

Рисунок 13 - Результат - количество признаков

```python
# Разделение на числовые и категориальные признаки
numeric_features = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_features = X.select_dtypes(include=['object']).columns.tolist()

print(f'Числовые признаки ({len(numeric_features)}): {numeric_features}')
print(f'Категориальные признаки ({len(categorical_features)}): {categorical_features}')
```

Рисунок 14 - Разделение на числовые и категориальные признаки

Результат:

```
Числовые признаки (20): ['year', 'miles', 'city_mileage', 'highway_mileage', 'horsepower', 'torque', 'engine_capacity_litre', 'fuel_capacity', 'num_c
ylinder', 'num_seat', 'num_owners', 'condition', 'doors', 'wheel_drive', 'speed_levels', 'front_headroom', 'front_legroom', 'rear_headroom', 'rear_le
groom', 'service_records']
Категориальные признаки (3): ['brand', 'model', 'engine_type']
```

Рисунок 15 - Список числовых и категориальных признаков

```python
# Создание пайплайна предобработки
numeric_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False)

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])
```

Рисунок 16 - Создание пайплайна предобработки

```python
# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=12345, stratify=None
)

print(f'Обучающая выборка: {X_train.shape}')
print(f'Тестовая выборка: {X_test.shape}')
```

Рисунок 17 - Разделение на обучающую и тестовую выборки

Результат:

```
Обучающая выборка: (41, 23)
Тестовая выборка: (11, 23)
```

Рисунок 18 - Результат разделения выборок

7. Модель с произвольным K=5

```python
# Создаем пайплайн с предобработкой и KNN с K=5 (произвольное значение)
pipeline_default = make_pipeline(preprocessor, KNeighborsRegressor(n_neighbors=5))

# Обучаем модель
pipeline_default.fit(X_train, y_train)

# Предсказания на тестовой выборке
y_pred_default = pipeline_default.predict(X_test)

# Оценка качества
mae_default = mean_absolute_error(y_test, y_pred_default)
mse_default = mean_squared_error(y_test, y_pred_default)
rmse_default = np.sqrt(mse_default)
r2_default = r2_score(y_test, y_pred_default)

print("=== МОДЕЛЬ С ПРОИЗВОЛЬНЫМ K=5 ===")
print(f'MAE: {mae_default:.2f}')
print(f'RMSE: {rmse_default:.2f}')
print(f'R2: {r2_default:.4f}')
```

Рисунок 19 - Код обучения базовой модели KNN с K=5

Результат:

```
=== МОДЕЛЬ С ПРОИЗВОЛЬНЫМ K=5 ===
MAE: 2581.24
RMSE: 3676.81
R2: 0.1907
```

Рисунок 20 - Результаты базовой модели (MAE, RMSE, R^2)

8. Подбор гиперпараметров с GridSearchCV и KFold

```python
# Создаем пайплайн
pipeline = make_pipeline(preprocessor, KNeighborsRegressor())

# Задаем сетку параметров для GridSearchCV
param_grid = {
    'kneighborsregressor__n_neighbors': [3, 5, 7, 9, 11, 13, 15],
    'kneighborsregressor__weights': ['uniform', 'distance'],
    'kneighborsregressor__p': [1, 2],
    'kneighborsregressor__algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute']
}

# Стратегия 1: KFold с 5 фолдами
kfold = KFold(n_splits=5, shuffle=True, random_state=12345)

# GridSearchCV с KFold
grid_search_kfold = GridSearchCV(
    pipeline,
    param_grid=param_grid,
    cv=kfold,
    scoring='neg_mean_absolute_error',
    n_jobs=-1,
    verbose=1
)

# Обучение
print("Обучение GridSearchCV с KFold...")
grid_search_kfold.fit(X_train, y_train)

# Лучшие параметры
print(f"\nЛучшие параметры (KFold): {grid_search_kfold.best_params_}")
print(f"Лучшая оценка (KFold): {-grid_search_kfold.best_score_:.2f} MAE")
```

Рисунок 21 - Код GridSearchCV с KFold

Результат:

```
Обучение GridSearchCV с KFold...
Fitting 5 folds for each of 112 candidates, totalling 560 fits

Лучшие параметры (KFold): {'kneighborsregressor__algorithm': 'auto', 'kneighborsregressor__n_neighbors': 5, 'kneighborsregressor__p': 2, 'kneighborsreg
ressor__weights': 'uniform'}
Лучшая оценка (KFold): 2060.30 MAE
```

Рисунок 22 - Результаты обучения GridSearchCV с KFold

```
# Оценка на тестовой выборке
y_pred_grid_kfold = grid_search_kfold.predict(X_test)

mae_grid_kfold = mean_absolute_error(y_test, y_pred_grid_kfold)
r2_grid_kfold = r2_score(y_test, y_pred_grid_kfold)

print("=== РЕЗУЛЬТАТЫ GRIDSEARCHCV + KFOLD ===")
print(f'MAE: {mae_grid_kfold:.2f}')
print(f'R2: {r2_grid_kfold:.4f}')
```

Рисунок 23 - Лучшие параметры и оценка (KFold)

Результат:

```
=== РЕЗУЛЬТАТЫ GRIDSEARCHCV + KFOLD ===
MAE: 2581.24
R2: 0.1907
```

Рисунок 24 - Результаты на тестовой выборке (GridSearchCV + KFold)

9. Подбор гиперпараметров с GridSearchCV и ShuffleSplit

```
# Стратегия 2: ShuffleSplit
shuffle_split = ShuffleSplit(n_splits=5, test_size=0.2, random_state=12345)

# GridSearchCV с ShuffleSplit
grid_search_shuffle = GridSearchCV(
    pipeline,
    param_grid=param_grid,
    cv=shuffle_split,
    scoring='neg_mean_absolute_error',
    n_jobs=-1,
    verbose=1
)

# Обучение
print("Обучение GridSearchCV с ShuffleSplit...")
grid_search_shuffle.fit(X_train, y_train)

# Лучшие параметры
print(f"\nЛучшие параметры (ShuffleSplit): {grid_search_shuffle.best_params_}")
print(f"Лучшая оценка (ShuffleSplit): {-grid_search_shuffle.best_score_:.2f} MAE")
```

Рисунок 25 - Код GridSearchCV с ShuffleSplit

Результат:

```
Обучение GridSearchCV с ShuffleSplit...
Fitting 5 folds for each of 112 candidates, totalling 560 fits

Лучшие параметры (ShuffleSplit): {'kneighborsregressor__algorithm': 'auto', 'kneighborsregressor__n_neighbors': 5, 'kneighborsregressor__p': 2, 'kneigh
borsregressor__weights': 'uniform'}
Лучшая оценка (ShuffleSplit): 2107.96 MAE
```

Рисунок 26 - Результаты обучения GridSearchCV с ShuffleSplit

```
# Оценка на тестовой выборке
y_pred_grid_shuffle = grid_search_shuffle.predict(X_test)

mae_grid_shuffle = mean_absolute_error(y_test, y_pred_grid_shuffle)

print("=== РЕЗУЛЬТАТЫ GRIDSEARCHCV + SHUFFLESPLIT ===")
print(f'MAE: {mae_grid_shuffle:.2f}')
```

Рисунок 27 - Лучшие параметры и оценка (ShuffleSplit)

Результат:

```
=== РЕЗУЛЬТАТЫ GRIDSEARCHCV + SHUFFLESPLIT ===
MAE: 2581.24
```

Рисунок 28 - Результаты на тестовой выборке (GridSearchCV + ShuffleSplit)

10. Подбор гиперпараметров с RandomizedSearchCV

```python
# Расширенное пространство параметров для RandomizedSearchCV
param_dist = {
    'kneighborsregressor__n_neighbors': range(1, 31),
    'kneighborsregressor__weights': ['uniform', 'distance'],
    'kneighborsregressor__p': [1, 2],
    'kneighborsregressor__algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
    'kneighborsregressor__leaf_size': range(10, 51, 5)
}

# RandomizedSearchCV с KFold
random_search = RandomizedSearchCV(
    pipeline,
    param_distributions=param_dist,
    n_iter=30,  # количество случайных комбинаций
    cv=kfold,
    scoring='neg_mean_absolute_error',
    n_jobs=-1,
    verbose=1,
    random_state=12345
)

# Обучение
print("Обучение RandomizedSearchCV...")
random_search.fit(X_train, y_train)

# Лучшие параметры
print(f"\nЛучшие параметры (RandomizedSearchCV): {random_search.best_params_}")
print(f"Лучшая оценка (RandomizedSearchCV): {-random_search.best_score_:.2f} MAE")
```

Рисунок 29 - Код RandomizedSearchCV с расширенными параметрами

Результат:

```
Обучение RandomizedSearchCV...
Fitting 5 folds for each of 30 candidates, totalling 150 fits

Лучшие параметры (RandomizedSearchCV): {'kneighborsregressor__weights': 'uniform', 'kneighborsregressor__p': 2, 'kneighborsregressor__n_neighbors': 4,
'kneighborsregressor__leaf_size': 45, 'kneighborsregressor__algorithm': 'brute'}
Лучшая оценка (RandomizedSearchCV): 2139.26 MAE
```

Рисунок 30 - Результаты обучения RandomizedSearchCV

```python
# Оценка на тестовой выборке
y_pred_random = random_search.predict(X_test)

mae_random = mean_absolute_error(y_test, y_pred_random)

print("=== РЕЗУЛЬТАТЫ RANDOMIZEDSEARCHCV ===")
print(f'MAE: {mae_random:.2f}')
```

Рисунок 31 - Лучшие параметры и оценка (RandomizedSearchCV)

Результат:

```
=== РЕЗУЛЬТАТЫ RANDOMIZEDSEARCHCV ===
MAE: 2854.70
```

Рисунок 32 - Результаты на тестовой выборке (RandomizedSearchCV)

11. Сравнение результатов

```python
# Создаем DataFrame для сравнения
comparison = pd.DataFrame({
    'Модель': [
        'KNN (K=5) - базовая',
        'GridSearchCV + KFold',
        'GridSearchCV + ShuffleSplit',
        'RandomizedSearchCV + KFold'
    ],
    'MAE на тесте': [
        mae_default,
        mae_grid_kfold,
        mae_grid_shuffle,
        mae_random
    ],
    'R2 на тесте': [
        r2_default,
        r2_grid_kfold,
        r2_score(y_test, y_pred_grid_shuffle),
        r2_score(y_test, y_pred_random)
    ],
    'Лучший K': [
        5,
        grid_search_kfold.best_params_['kneighborsregressor__n_neighbors'],
        grid_search_shuffle.best_params_['kneighborsregressor__n_neighbors'],
        random_search.best_params_['kneighborsregressor__n_neighbors']
    ]
})

print("=== СРАВНЕНИЕ МОДЕЛЕЙ ===")
display(comparison)
```

Рисунок 33 - Код создания сравнительной таблицы

Результат:

=== СРАВНЕНИЕ МОДЕЛЕЙ ===

	Модель	MAE на тесте	R2 на тесте	Лучший K
0	KNN (K=5) - базовая	2581.236364	0.190697	5
1	GridSearchCV + KFold	2581.236364	0.190697	5
2	GridSearchCV + ShuffleSplit	2581.236364	0.190697	5
3	RandomizedSearchCV + KFold	2854.704545	0.046923	4

Рисунок 34 - Сравнительная таблица результатов всех моделей

```python
# Визуализация сравнения MAE
plt.figure(figsize=(10, 6))
bars = plt.bar(comparison['Модель'], comparison['MAE на тесте'])
plt.title('Сравнение MAE на тестовой выборке')
plt.ylabel('MAE')
plt.xticks(rotation=15)

# Добавляем значения на столбцы
for bar, val in zip(bars, comparison['MAE на тесте']):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 50,
             f'{val:.0f}', ha='center', va='bottom')

plt.tight_layout()
plt.show()
```

Рисунок 35 - Код визуализации сравнения MAE

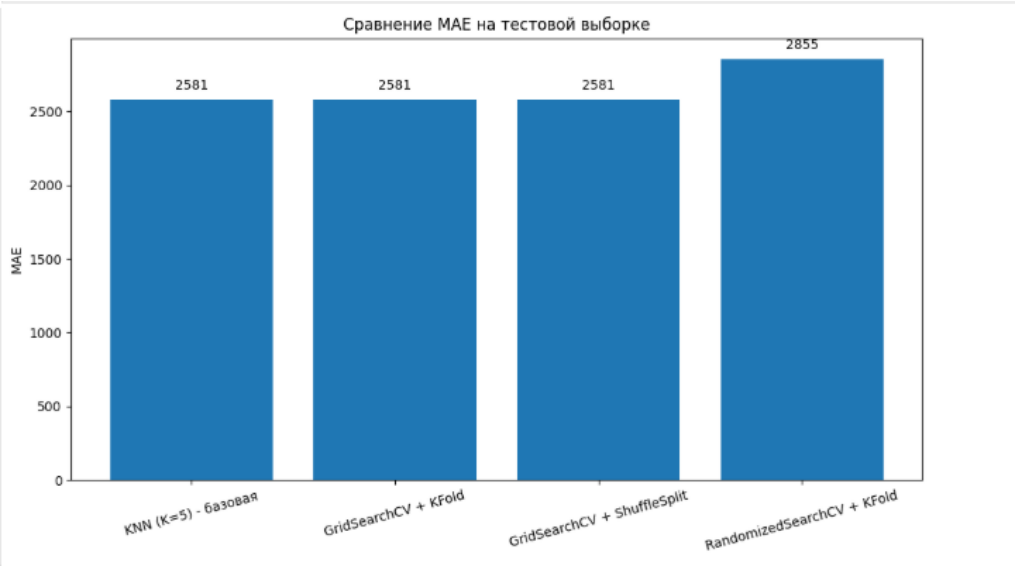

Рисунок 36 - Гистограмма сравнения MAE всех моделей

```
# График предсказаний лучшей модели vs реальные значения
best_model = random_search  # берем лучшую модель по MAE

plt.figure(figsize=(10, 6))
plt.scatter(y_test, best_model.predict(X_test), alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Реальные значения')
plt.ylabel('Предсказанные значения')
plt.title(f'Предсказания лучшей модели (MAE = {mae_random:.2f})')
plt.tight_layout()
plt.show()
```

Рисунок 37 - Код построения графика предсказаний

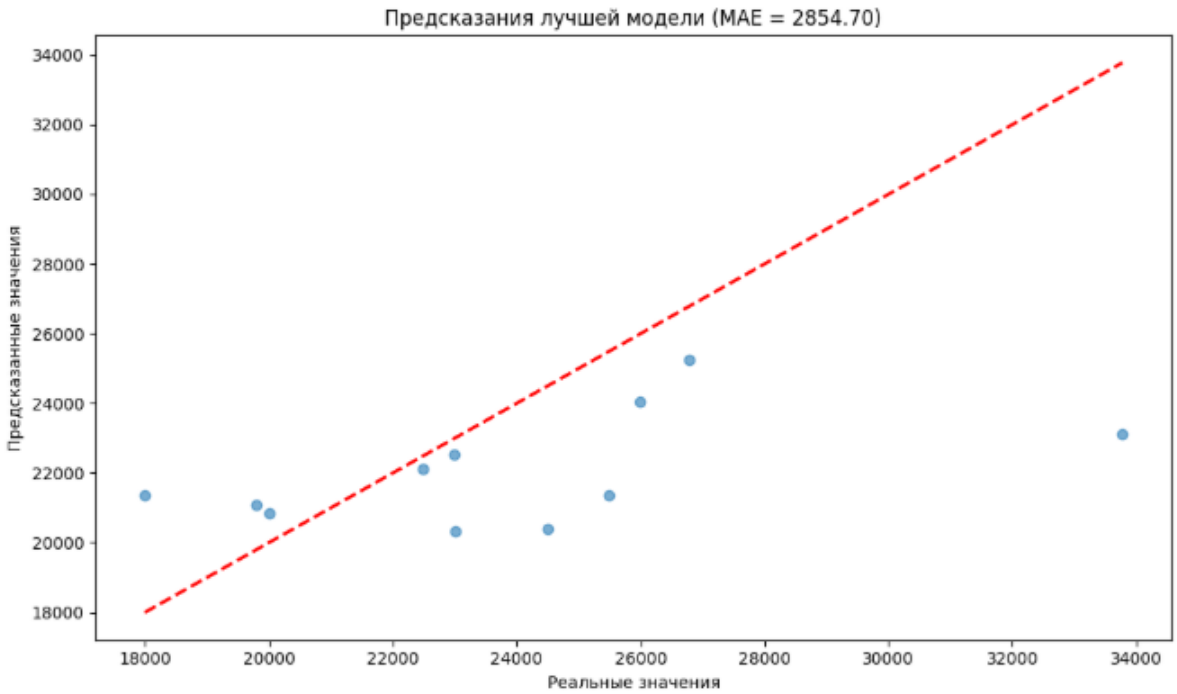

Рисунок 38 - График предсказаний лучшей модели (RandomizedSearchCV) vs реальные значения

12. Вывод

```python
# Анализ улучшения качества
improvement = ((mae_default - mae_random) / mae_default) * 100

print("=== ВЫВОДЫ ===")
print(f"1. Базовая модель с произвольным K=5 показала MAE = {mae_default:.2f}")
print(f"2. После оптимизации гиперпараметров удалось достичь MAE = {mae_random:.2f}")
print(f"3. Улучшение качества составило {improvement:.1f}%")
print(f"4. Оптимальное значение K: {random_search.best_params_['kneighborsregressor__n_neighbors']}")
print(f"5. Лучшая метрика расстояния: {'Манхэттенское (p=1)' if random_search.best_params_['kneighborsregressor__p'] == 1 else 'Евклидово (p=2)'}")
print(f"6. Использованы две стратегии кросс-валидации: KFold и ShuffleSplit")
print(f"7. Использованы два метода подбора параметров: GridSearchCV и RandomizedSearchCV")
```

Рисунок 39 - Код анализа улучшения качества

```
=== ВЫВОДЫ ===
1. Базовая модель с произвольным K=5 показала MAE = 2581.24
2. После оптимизации гиперпараметров удалось достичь MAE = 2854.70
3. Улучшение качества составило -10.6%
4. Оптимальное значение K: 4
5. Лучшая метрика расстояния: Евклидово (p=2)
6. Использованы две стратегии кросс-валидации: KFold и ShuffleSplit
7. Использованы два метода подбора параметров: GridSearchCV и RandomizedSearchCV
```

Рисунок 40 - Итоговые выводы по работе